Lab 1 实验报告

一、实验目的

在禁止控制流、比较运算符与常规算术运算的条件下，仅用位运算完成 19 个函数。

二、实验内容与约束

整数题只允许 `0`–`255` 的常量、函数参数与 `int` 局部变量，以及 `! ~ & ^ | + << >>`；禁止 `if`/`while` 等控制流、`&& || - * / %`、全部比较运算符、类型转换、宏与函数调用，并且每题有严格的操作数上限，其中 P2 进一步限制为仅可用 `~` 与 `&`。 浮点题（P15–P17）放宽为 `int`/`unsigned` 的任意整数常量与全部整数运算，也允许 `if`/`while`，但仍禁止浮点类型、浮点常量与类型转换。 两类题目都不得使用 `int`、`unsigned` 以外的数据类型（数组、结构体、联合体）。

三、逐题实现与分析

编号	函数	输入约束	合法运算符	最大操作数	分值
P1	`signMask()`	无参数	通用整数运算	2	1
P2	`bitXor(x,y)`	任意 `int`	仅 `~ &`	8	2
P3	`negativePart(x)`	任意 `int`	通用整数运算	6	3
P4	`copyByteWithin(x,src,dst)`	`src,dst` 为 0–3	通用整数运算	12	4
P5	`logicalShift(x,n)`	`0<=n<=31`	通用整数运算	20	4
P6	`swapNibblePairs(x)`	任意 `int`	通用整数运算	18	4
P7	`secondLowestZeroBit(x)`	任意 `int`	通用整数运算	8	4
P8	`oddParity(x)`	任意 `int`	通用整数运算	56	5
P9	`rotateRightBits(x,n)`	`n>=0`，按 32 取模	通用整数运算	16	5
P10	`roundEvenPow2(x,n)`	`0<=x<=0x3fffffff`，`1<=n<=16`	通用整数运算	24	5

编号	函数	输入约束	合法运算符	最大操作数	分值
P11	`midpointTowardFirst(x,y)`	任意 `int`	通用整数运算	32	5
P12	`isBetweenEitherOrder(x,a,b)`	任意 `int`	通用整数运算	48	7
P13	`mul5Sat(x)`	任意 `int`	通用整数运算	30	7
P14	`classifyAdd3(x,y,z)`	任意 `int`	通用整数运算	52	7
P15	`floatScaleThreeHalves(uf)`	任意单精度位模式	浮点题规则	60	7
P16	`floatRoundEven(uf)`	任意单精度位模式	浮点题规则	65	10
P17	`float_i2f(x)`	任意 `int`	浮点题规则	40	10
P18	`bitCount(x)`	任意 `int`	通用整数运算	40	10
P19	`bitReverse(x)`	任意 `int`	通用整数运算	34	10

P1 `signMask()` — 返回最高位掩码，`1 << 31` 的位模式即 `0x80000000`。

P2 `bitXor(x,y)` — 只许 `~` `&`。由 `x ^ y = (x | y) & ~(x & y)` 出发，再用德摩根律把 `x | y` 写成 `~(~x & ~y)`，得 `~(~x & ~y) & ~(x & y)`。

P3 `negativePart(x)` — 用算术右移把符号位广播成整字掩码：`x < 0` 时 `x >> 31` 为全 1，否则为 0。再由补码取负恒等式 `-x = ~x + 1`（对 `INT_MIN` 同样成立）与之相与，条件成立输出 `-x`，否则被清零。

P4 `copyByteWithin(x,src,dst)` — 移位量是变量，字节位置需现场算：`x >> (src << 3) & 0xFF` 取出源字节（`& 0xFF` 顺带清掉符号扩展位），`~(0xFF << (dst << 3)) & x` 清空目标字节，最后把源字节左移回 `dst` 处相或。`src`、`dst` 最大为 3，移位量 24 仍在合法范围内。

P5 `logicalShift(x,n)` — C 对 `int` 的 `>>` 是算术右移、会补符号位，所以先造"高 `n` 位为 0、低位全 1"的掩码 `~(((1 << 31) >> n) << 1)`，再与 `x >> n` 相与把它抹掉。`n = 0` 时 `<<1` 恰好把最高位丢出、取反得全 1，自然退化为返回原值。

P6 `swapNibblePairs(x)` — 造周期性掩码 `0x0F0F0F0F`（由 4 个不超过 `0xFF` 的常量拼成），`(x >> 4) & mask` 取各字节的高半字节并下移，`(x & mask) << 4` 取低半字节并上移，两者相或。`& mask` 只保留每字节低 4 位，算术右移填入的符号位被一并清除。

P7 `secondLowestZeroBit(x)` — `~x & (x + 1)` 恰好标记 `x` 中最低的那个 0 位：`x + 1` 的进位链把该位翻成 1、更低位归 0，而 `~x` 只在该位及更高位为 1。要做"第二个"，先把这一位补成 1，再对新的 `x` 重复同一运算即可；`x` 全 1 时结果为 0。

P8 `oddParity(x)` — 折叠异或：把高 16 位与低 16 位异或，依次再对折 8、4、2、1 位，最低位就是全部 32 位的异或和（为 0 表示 1 的个数为偶数）。题目要求偶数个 1 返回 1，故最后取 `~x & 1`。

P9 `rotateRightBits(x,n)` — 循环右移 = 逻辑右移 `n` 位 与 左移 `32-n` 位 的并集。`n & 31` 实现按 32 取模；右半部分复用 P5 的掩码；左移量写成 `(~n + 1) & 31`（即 `(-n) mod 32`），避免出现 32 这个非法移位量，`n = 0` 时自然退化为不移。

P10 `roundEvenPow2(x,n)` — 先按普通四舍五入算出 `(x + half) >> n`，再修正中点：余数 `r = x & (2^n - 1)`，当 `r == half` 时正好落在两个倍数中间，此时若原商 `y = x >> n` 为偶数（`!(y & 1)`），说明进位后会落到奇数商上，需要减 1 回到偶数。

P11 `midpointTowardFirst(x,y)` — 用 `(x & y) + ((x ^ y) >> 1)` 求下取整中点（不溢出），`(x ^ y) & 1` 判断是否带 `.5`；`x > y` 不能用比较运算符，改为异号时看符号位、同号时看 `x - y` 的符号位（同号相减不会溢出）。中点带 `.5` 且 `x > y` 时取 `mid + 1`，否则取 `mid`。

```
int midpointTowardFirst(int x, int y) {
  int mid = (x & y) + ((x ^ y) >> 1);
  int is_odd = (x ^ y) & 1;

  int sign_x = x >> 31;
  int sign_y = y >> 31;
  int diff_sign = sign_x ^ sign_y;

  int x_minus_y = x + ~y + 1;
  int same_sign_ge = !(x_minus_y >> 31);
  int diff_sign_ge = !sign_x;

  int x_ge_y = (diff_sign & diff_sign_ge) | (~diff_sign & same_sign_ge);

  return mid + (is_odd & x_ge_y);
}
```

P12 `isBetweenEitherOrder(x,a,b)` — 区间判定等价于 `(a<=x && x<=b) || (b<=x && x<=a)`，难点是在不溢出、不用比较运算符的前提下实现 `u <= v`：异号时它等价于 `u < 0`（看符号位）；同号时做差不会溢出，看 `(u - v) >> 31` 是否为 1 或两者是否相等。四个方向各造一个谓词后拼接即可，返回值用 `& 1` 归一化。

```
int isBetweenEitherOrder(int x, int a, int b) {
  int sa = a >> 31;
  int sb = b >> 31;
  int sx = x >> 31;

  int nx = ~x;

  int da = a + nx + 1;        // a - x
  int db = b + nx + 1;        // b - x
  int dxa = ~da + 1;          // x - a
  int dxb = ~db + 1;          // x - b

  int dax = sa ^ sx;
```

```
  int dbx = sb ^ sx;

  int eq_ax = !(a ^ x);
  int eq_bx = !(b ^ x);

  int le_ax = (dax & sa) | (~dax & ((da >> 31) | eq_ax));
  int le_bx = (dbx & sb) | (~dbx & ((db >> 31) | eq_bx));
  int le_xa = (dax & sx) | (~dax & ((dxa >> 31) | eq_ax));
  int le_xb = (dbx & sx) | (~dbx & ((dxb >> 31) | eq_bx));

  return ((le_ax & le_xb) | (le_bx & le_xa)) & 1;
}
```

P13 `mul5Sat(x)` — 用无分支绝对值 `(x + (x >> 31)) ^ (x >> 31)` 取 `|x|`；溢出条件 `|x| > INT_MAX / 5 = 0x19999999` 用差值符号位判断（正负两侧共用同一阈值，因为 `5|x|` 触及 `INT_MAX` 与触及 `2^31` 只差一个单位）；饱和值按符号在 `INT_MIN` / `INT_MAX` 中选择；正常结果 `x*5 = (x << 2) + x`；最后用 `overflow` 展开成全 1 / 全 0 的掩码在饱和值与正常结果之间二选一。

```
int mul5Sat(int x) {
  int sign = x >> 31;
  int abs_x = (x + sign) ^ sign;
  int thresh = (0x19 << 24) | (0x99 << 16) | (0x99 << 8) | 0x99; // 0x19999999
  int diff = abs_x + ~thresh + 1;
  int overflow = !((diff >> 31) | (!diff));

  int sign_bit = 1 << 31;
  int max = ~sign_bit;
  int min = sign_bit;
  int sat = (sign & min) | (~sign & max);

  int result = (x << 2) + x;
  int ov_mask = ~overflow + 1;
  return (ov_mask & sat) | (~ov_mask & result);
}
```

P14 `classifyAdd3(x,y,z)` — 把三数求和拆成两次 32 位加法，同时统计两个量：负数（符号位为 1）的个数 `K`，以及两次加法实际产生的进位次数 `C`（进位用 `(a & b) | ((a | b) & ~s)` 的符号位提取）。`D = C - K` 的正负即净进位 / 净借位的方向：`D > 0` 返回 1，`D < 0` 返回 -1。`D == 0` 时没有净溢出，但若此时中间和 `s2` 为负，说明三个正数相加后符号翻转，仍属正向溢出。

```
int classifyAdd3(int x, int y, int z) {
  int kx = (x >> 31) & 1;
  int ky = (y >> 31) & 1;
  int kz = (z >> 31) & 1;
  int K = kx + ky + kz;

  int s1 = x + y;
  int c1 = ((x & y) | ((x | y) & ~s1)) >> 31 & 1;
```

```
  int s2 = s1 + z;
  int c2 = ((s1 & z) | ((s1 | z) & ~s2)) >> 31 & 1;
  int C = c1 + c2;

  int D = C + (~K + 1);
  int D_neg = D >> 31;
  int D_zero = !D;
  int D_pos = !D_neg & !D_zero;

  int s2_sign = (s2 >> 31) & 1;

  int pos = D_pos | (D_zero & s2_sign);
  int D_plus_1 = D + 1;
  int D_is_minus_1 = !D_plus_1;
  int D_le_minus_2 = D_neg & !D_is_minus_1;
  int neg = D_le_minus_2 | (D_is_minus_1 & !s2_sign);

  return pos + (~neg + 1);
}
```

P15 `floatScaleThreeHalves(uf)` — 把乘 1.5 转化为尾数的整数乘法：拆出 `sign / exp / frac`，`NaN` / `±0` 原样返回，规格化与非规格化统一成 `(mant, E)`；`V = mant * 3` 之后由最高有效位 `p` 反推新阶码 `E + p - 24`；被丢弃的低位与半程比较，按 round-half-to-even 舍入，尾数进位则阶码加一，阶码到 255 时返回带符号的无穷。`exp_new <= 0` 时进入非规格化区间，用一次整数右移完成该区间的偶数舍入。

```
unsigned floatScaleThreeHalves(unsigned uf) {
    unsigned sign = uf & 0x80000000;
    unsigned exp = (uf >> 23) & 0xFF;
    unsigned frac = uf & 0x7FFFFF;

    if (exp == 255) {
        return uf;
    }

    if (exp == 0 && frac == 0) {
        return uf;
    }

    unsigned mant;
    int E;
    if (exp == 0) {
        mant = frac;
        E = -126;
    } else {
        mant = (1 << 23) | frac;
        E = exp - 127;
    }

    unsigned V = mant * 3;

    int p = 31;
```

```
    while (p >= 0 && !(V & (1u << p))) {
        p--;
    }

    int E_new = E + p - 24;
    int exp_new = E_new + 127;

    if (exp_new >= 255) {
        return sign | 0x7F800000;
    }

    unsigned frac_new;
    if (exp_new > 0) {
        int shift = p - 23;
        unsigned high = V >> shift;
        frac_new = high & 0x7FFFFF;

        if (shift > 0) {
            unsigned dropped = V & ((1u << shift) - 1);
            unsigned half = 1u << (shift - 1);
            if (dropped > half || (dropped == half && (frac_new & 1))) {
                frac_new++;
                if (frac_new == 0x800000) {
                    frac_new = 0;
                    exp_new++;
                    if (exp_new >= 255) {
                        return sign | 0x7F800000;
                    }
                }
            }
        }
    } else {
        frac_new = (V >> 1) + ((V & 1) & ((V >> 1) & 1));
        exp_new = 0;
    }

    return sign | (exp_new << 23) | frac_new;
}
```

P16 `floatRoundEven(uf)` — 按数量级分段：`exp < 126` 时 `|f| < 0.5`，结果必为 0，返回 `sign` 以保留 `-0`；`exp == 126` 时恰好 `±0.5` 取偶返回 `±0`，否则返回 `±1`；`E >= 23` 时尾数 23 位全是整数部分，值本身已是整数，原样返回；其余情况取 `M = (1 << 23) | frac`，用 `shift = 23 - E` 分出整数位 `I` 与余数位 `F`，按 round-half-to-even 决定是否 `I++`，最后重新归一化（`p == 23` 表示恰好进位到 `2^23`）。`NaN` 与 `Inf` 原样返回。

```
unsigned floatRoundEven(unsigned uf) {
    unsigned sign = uf & 0x80000000;
    unsigned exp = (uf >> 23) & 0xFF;
    unsigned frac = uf & 0x7FFFFF;

    if (exp == 255) return uf;
```

```
    if (exp == 0 && frac == 0) return uf;

    if (exp < 127) {
        if (exp < 126) {
            return sign;
        } else {
            if (frac == 0) {
                return sign;
            } else {
                return sign | 0x3F800000;
            }
        }
    }

    int E = exp - 127;
    if (E >= 23) return uf;

    unsigned M = (1 << 23) | frac;
    int shift = 23 - E;
    unsigned I = M >> shift;
    unsigned F = M & ((1 << shift) - 1);
    unsigned half = 1 << (shift - 1);

    if (F > half || (F == half && (I & 1))) {
        I++;
    }

    int p = 31;
    while (p >= 0 && !(I & (1u << p))) {
        p--;
    }

    if (p == 23) {
        return sign | (150 << 23);
    } else {
        unsigned new_exp = 127 + p;
        unsigned new_frac = (I << (23 - p)) & 0x7FFFFF;
        return sign | (new_exp << 23) | new_frac;
    }
}
```

P17 `float_i2f(x)` — `x == 0` 返回 `+0`；保存符号位后求绝对值（`~x + 1`，`INT_MIN` 也不溢出）；循环右移找到最高有效位得阶 `e`，阶码为 `e + 127`；`e <= 23` 时左移对齐到 23 位，否则右移丢弃 `e - 23` 位并按 round-half-to-even 舍入，尾数进位到 `0x800000` 时归零且阶码加一。

```
unsigned float_i2f(int x) {
  if (x == 0) return 0;

  unsigned sign = x & 0x80000000;
  unsigned ux = x;
  unsigned abs_x;
```

```
    if (x < 0) {
      abs_x = ~ux + 1;
    } else {
      abs_x = ux;
    }

    int e = 0;
    unsigned temp = abs_x;
    while (temp > 1) {
      temp >>= 1;
      e++;
    }

    unsigned exp = e + 127;
    unsigned mantissa;

    if (e <= 23) {
      mantissa = (abs_x << (23 - e)) & 0x7FFFFF;
    } else {
      int shift = e - 23;
      unsigned high = abs_x >> shift;
      unsigned m = high & 0x7FFFFF;
      unsigned mask = (1 << shift) - 1;
      unsigned dropped = abs_x & mask;
      unsigned half = 1 << (shift - 1);

      if (dropped > half || (dropped == half && (m & 1))) {
        m++;
        if (m == 0x800000) {
          m = 0;
          exp++;
        }
      }
      mantissa = m;
    }

    return sign | (exp << 23) | mantissa;
  }
```

P18 `bitCount(x)` — 分治并行计数（SWAR）：先在 2 位域内统计，再在 4、8、16 位域内合并，5 步完成。掩码 `0x55555555`、`0x33333333`、`0x0F0F0F0F` 等由不超过 `0xFF` 的小常量移位拼成；每步 `(count & mask) + ((count >> k) & mask)` 把相邻两域的和并进同一域、域宽翻倍且互不干扰。后几步域内和不会超出域宽，可以省掉一次掩码，只在最后截取。

P19 `bitReverse(x)` — 分治块交换，与 P18 互为对偶：依次交换 16、8、4、2、1 位块，5 步完成。掩码用 `mask ^= mask << k` 递推生成（`0x0000FFFF → 0x00FF00FF → 0x0F0F0F0F → 0x33333333 → 0x55555555`），每步套同一模板 `((x >> k) & mask) | ((x & mask) << k)`，比逐个手写常量更省操作数。

四、测试与验证

本次实验的验证在 WSL Ubuntu 的 32 位编译环境下完成。构建用 `make clean && make all`，生成 `btest`、`ishow`、`fshow`；因为 `btest` 不会在源文件修改后自动重编译，每次改动 `bits.c` 后都重新执行了这两条命令。

运算符合法性与操作数上限统一用 `./check_ops.py bits.c` 检查，它逐题打印"当前操作数 / 最大操作数"并调用 `dlc` 校验语法。仓库自带的 `dlc` 内置的是旧函数名、无法直接检查整份文件，所以由该脚本为每道新题选择同规则的代理函数名后再调用。

正确性上先用 `./btest -f 函数名` 单题调试，`-1`、`-2`、`-3` 用于指定前三个参数（如 `./btest -f roundEvenPow2 -1 5 -2 1`），依据失败样例打印的输入、实际结果与期望结果，判断问题属于符号、溢出、移位、舍入还是浮点特殊值，修正后再用 `./btest` 跑完整测试；调试过程中用 `./ishow`、`./fshow` 查看整数与浮点数的位表示辅助定位。

在 `~/Lab1` 下执行 `./btest`，19 题的 `Errors` 全为 0，总分 110/110：

编号	函数	错误数	得分
P1	`signMask`	0	1
P2	`bitXor`	0	2
P3	`negativePart`	0	3
P4	`copyByteWithin`	0	4
P5	`logicalShift`	0	4
P6	`swapNibblePairs`	0	4
P7	`secondLowestZeroBit`	0	4
P8	`oddParity`	0	5
P9	`rotateRightBits`	0	5
P10	`roundEvenPow2`	0	5
P11	`midpointTowardFirst`	0	5
P12	`isBetweenEitherOrder`	0	7
P13	`mul5Sat`	0	7
P14	`classifyAdd3`	0	7
P15	`floatScaleThreeHalves`	0	7
P16	`floatRoundEven`	0	10
P17	`float_i2f`	0	10
P18	`bitCount`	0	10
P19	`bitReverse`	0	10
—	**合计**	**0**	**110**

```
Total points: 110/110
```

五、关键总结

用掩码代替分支。 整数题禁用 `if` 与比较运算符，条件判断归结为四种写法：`x >> 31` 把符号位广播成全 1 / 全 0 掩码作无分支选择器（P3、P13）；`!(a ^ b)` 用异或零判等代替 `==`（P10、P12、P14）；`(a - b) >> 31` 在同号前提下比较大小（P11、P12、P13）；`(a >> 31) ^ (b >> 31)` 处理异号情形（P11、P12）。**同号才可安全做差**是反复出现的前提——异号相减会溢出，使符号位失真。

三个无溢出恒等式。 `abs(x) = (x + (x >> 31)) ^ (x >> 31)`、`-x = ~x + 1`、`(x & y) + ((x ^ y) >> 1)`，三者在 `INT_MIN` / `INT_MAX` 处都不产生未定义行为。

掩码的三种造法。 广播式：`1 << 31`、`~(((1 << 31) >> n) << 1)`（P5、P9）；周期性：用不超过 `0xFF` 的小常量移位拼出 `0x55555555`、`0x0F0F0F0F` 等（P6、P18、P19）；递推式：`mask ^= mask << k`（P19）。常量不能超过 255 这一限制，正是这些拼接技巧存在的原因。

分治（SWAR）思想。 P18 与 P19 是同一思想的正反两面，把 32 位拆成 2/4/8/16 位域逐步合并，5 步完成全部工作。

IEEE 754 的统一骨架。 P15–P17 共用同一套流程：拆 `sign / exp / frac` → 短路 `NaN`、`Inf` 与 `±0` → 把规格化与非规格化统一成 `(mant, E)` → 在尾数域内用整数运算 → round-half-to-even 舍入 → 处理尾数进位引起的阶码加一乃至溢出到无穷。负数只保管 `sign`，全部运算在绝对值域进行。其中"尾数进位 → 阶码加一 → 可能溢出到无穷"这条链在三个函数中都出现，是浮点实现最容易遗漏的边界。

报告结束